BÀI THỰC HÀNH FLUTTER

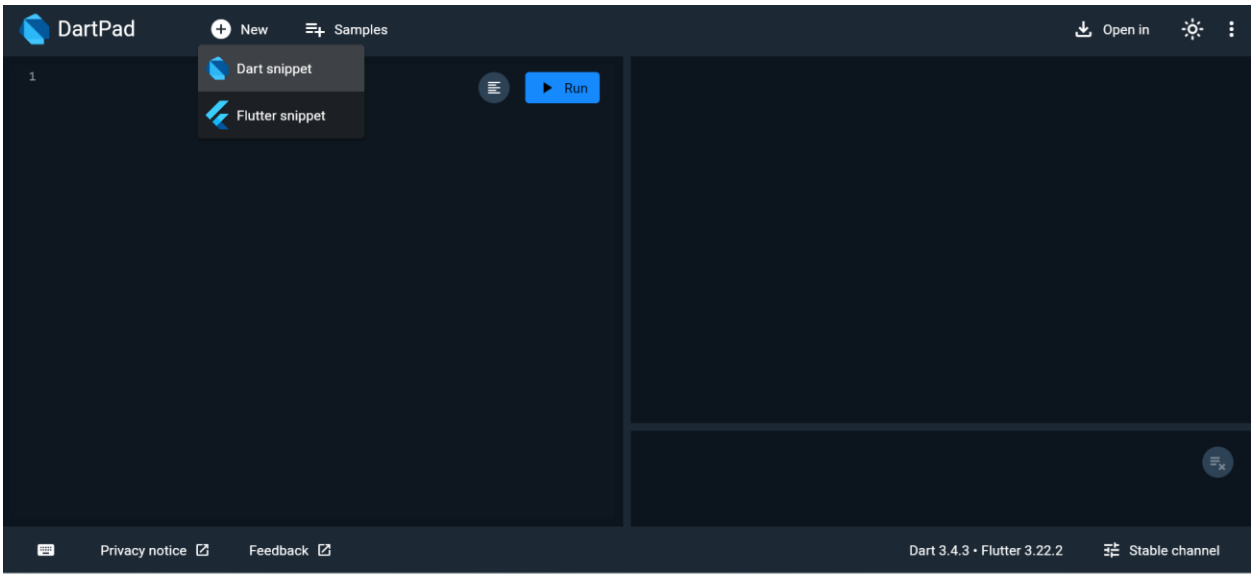
Contents

Môi trường: 1
Tạo ứng dụng Flutter với Dartpad: 1
Tạo ứng dụng Flutter với Android Studio 15
Bài 1: 24
Bài 2: 28
Bài 3: 31
Bài 4: 36
Bài 5: 43
Bài 6: 49

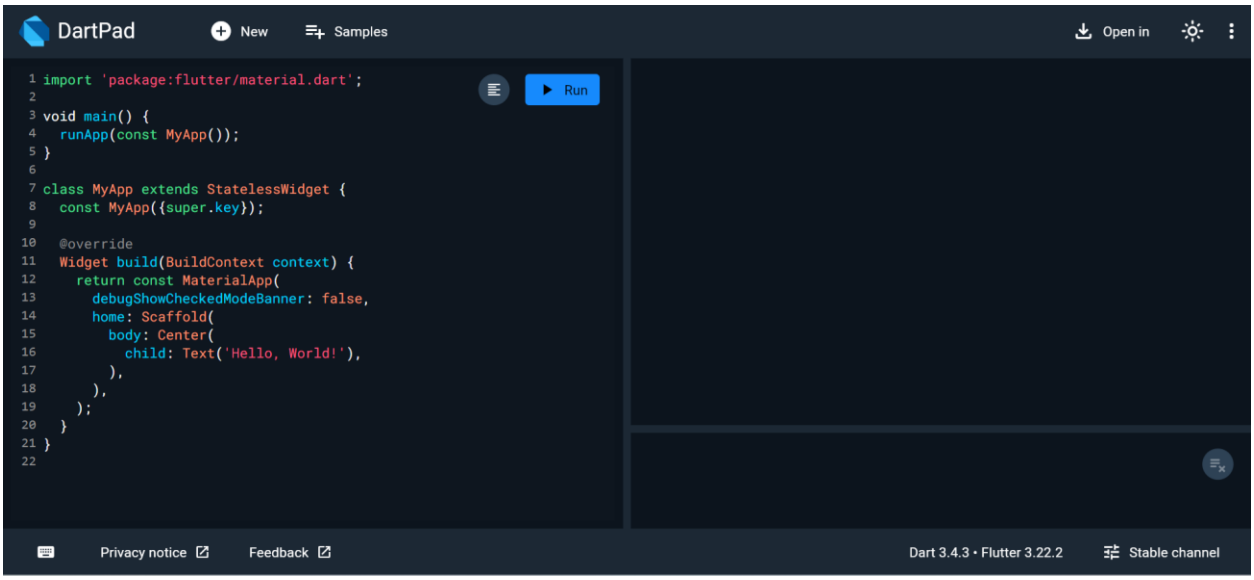
Môi trường:

Tạo ứng dụng Flutter với Dartpad:

Tạo ứng dụng Flutter bằng cách chọn New → Flutter snippnet



Ứng dụng tạo tự động sẽ có code như sau:



Giải thích code:

1. **Imports:** Đầu tiên, import gói flutter/material.dart, gói này cung cấp các tiện ích và thành phần cơ bản của Flutter. (cung cấp các widget và các công cụ thiết kế giao diện người dùng tuân theo ngôn ngữ thiết kế Material của Google.)

```
import 'package:flutter/material.dart';
```

2. **main():** Hàm main() khởi chạy ứng dụng bằng cách gọi runApp() với MyApp. Hàm main() là điểm khởi đầu của ứng dụng. Hàm runApp nhận vào một widget, ở đây là MyApp, và hiển thị nó lên màn hình. const MyApp() tạo một instance của MyApp với từ khóa const để tối ưu hóa hiệu năng và đảm bảo rằng widget này là bất biến.

```
void main() {
  runApp(MyApp());
}
```

3. **MyApp:** Đây là lớp chính của ứng dụng, kế thừa từ StatelessWidget.

```
class MyApp extends StatelessWidget {

  const MyApp({super.key});

  @override

  Widget build(BuildContext context) {
```

```
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: Scaffold(
        body: Center(
          child: Text('Hello, World!'),
        ),
      ),
    );
  }
}
```

- class MyApp extends StatelessWidget: MyApp là một lớp kế thừa từ StatelessWidget, nghĩa là trạng thái của nó không thay đổi trong suốt vòng đời của widget này.
- const MyApp({super.key});: Constructor của MyApp, sử dụng từ khóa const để tạo một instance bất biến của MyApp. super.key là tham số được truyền đến constructor của lớp cha StatelessWidget.
- @override: Chỉ thị cho trình biên dịch rằng phương thức build được định nghĩa lại từ lớp cha StatelessWidget.
- Widget build(BuildContext context): Phương thức build xây dựng và trả về cây widget mà Flutter hiển thị.

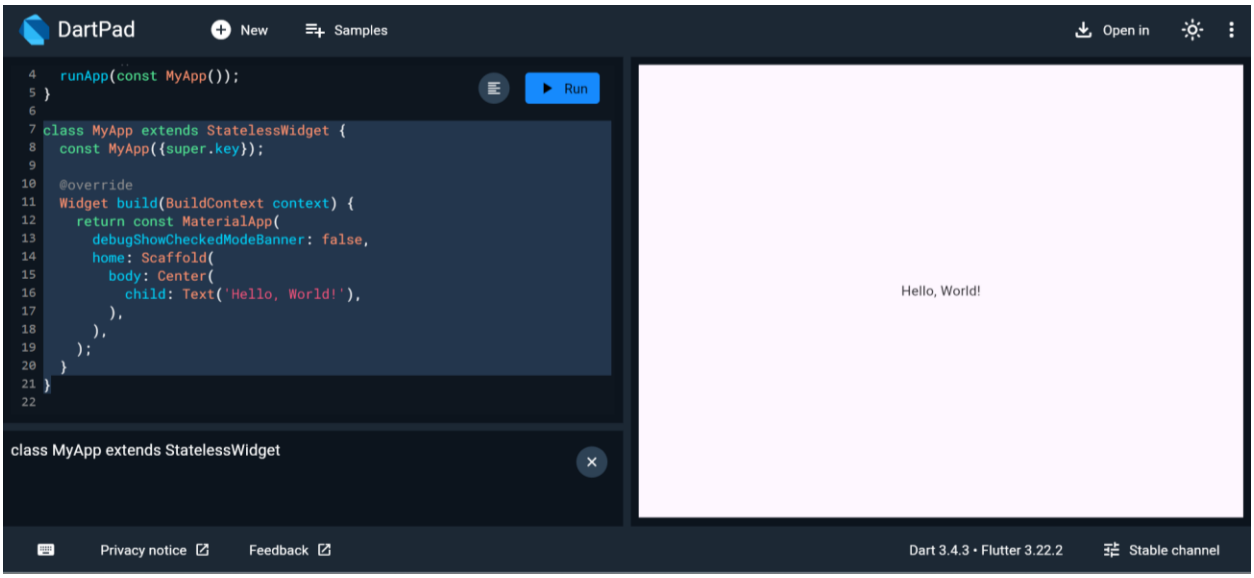
MaterialApp: Widget gốc của ứng dụng, cung cấp nhiều tính năng như điều hướng, chủ đề, và hỗ trợ đa ngôn ngữ. debugShowCheckedModeBanner: false loại bỏ biểu ngữ "DEBUG" ở góc trên bên phải khi chạy ứng dụng trong chế độ debug.

home: Scaffold: Xác định widget gốc của ứng dụng. Scaffold cung cấp một cấu trúc giao diện cơ bản như app bar, drawer, floating action button, và các thành phần cơ bản khác của giao diện Material Design.

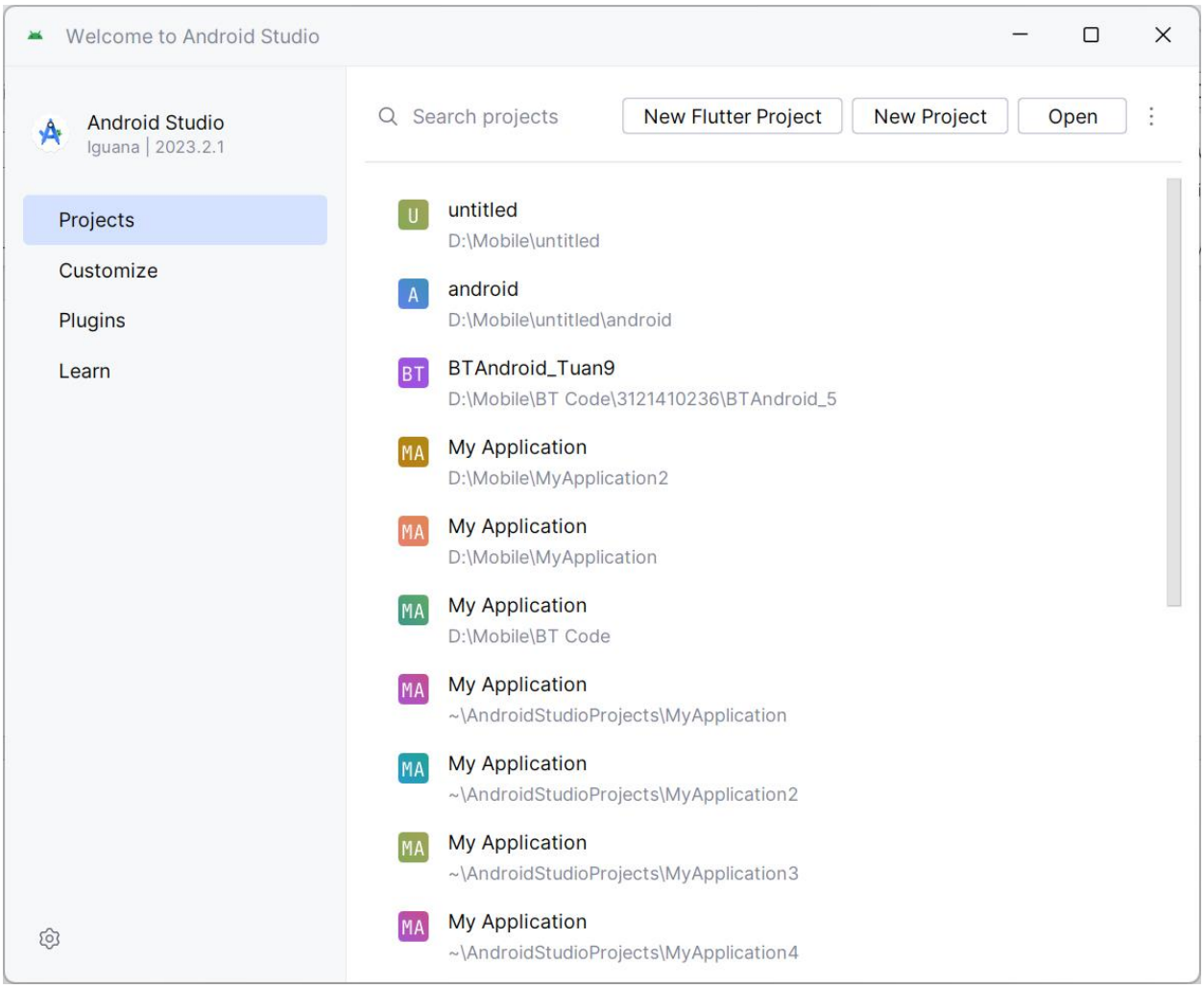
body: Center: Widget Center căn giữa con của nó theo cả chiều ngang và chiều dọc.

child: Text('Hello, World!'): Widget Text hiển thị một đoạn văn bản. Ở đây, đoạn văn bản là 'Hello, World!'.

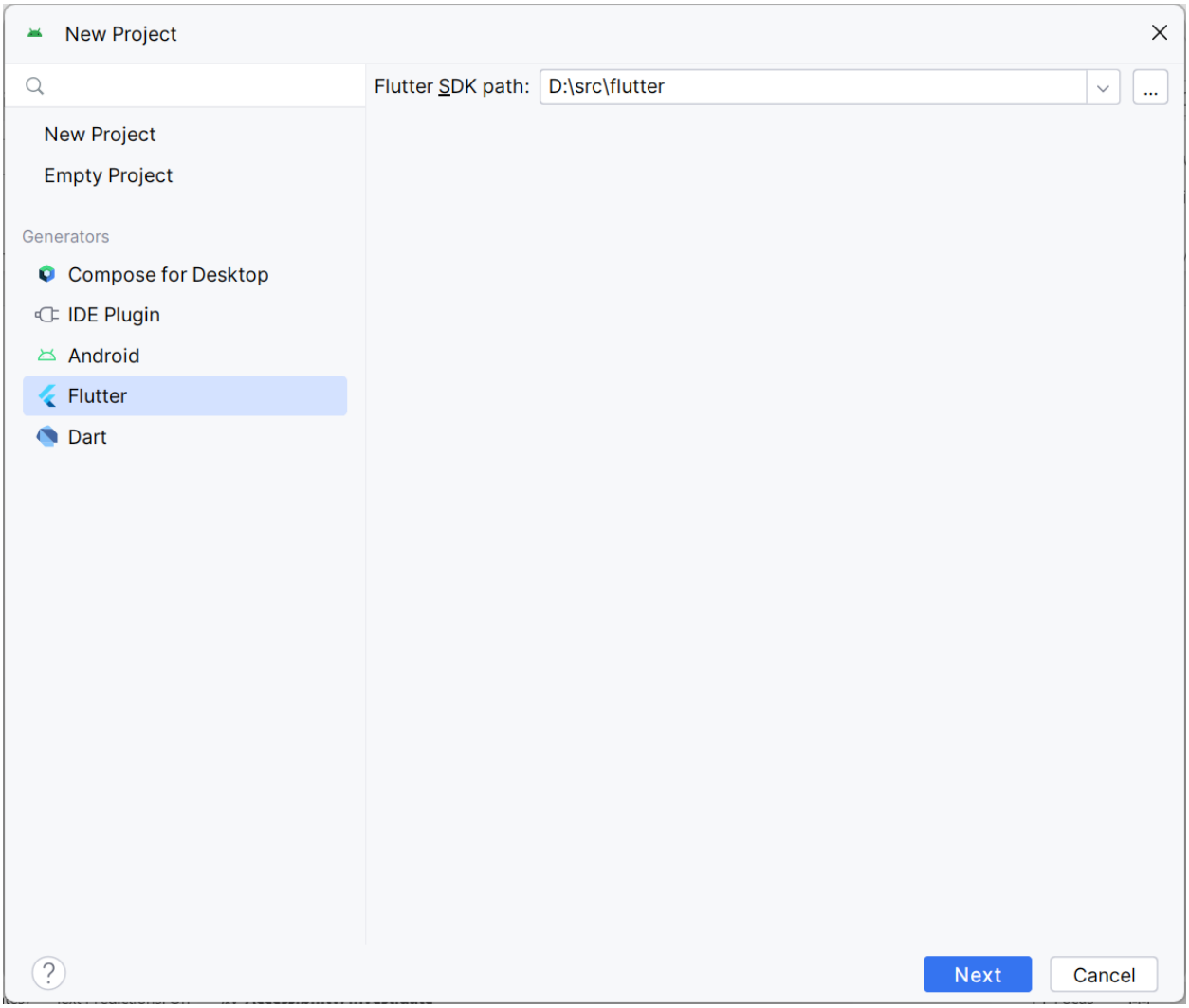
Bấm nút Run và xem kết quả:



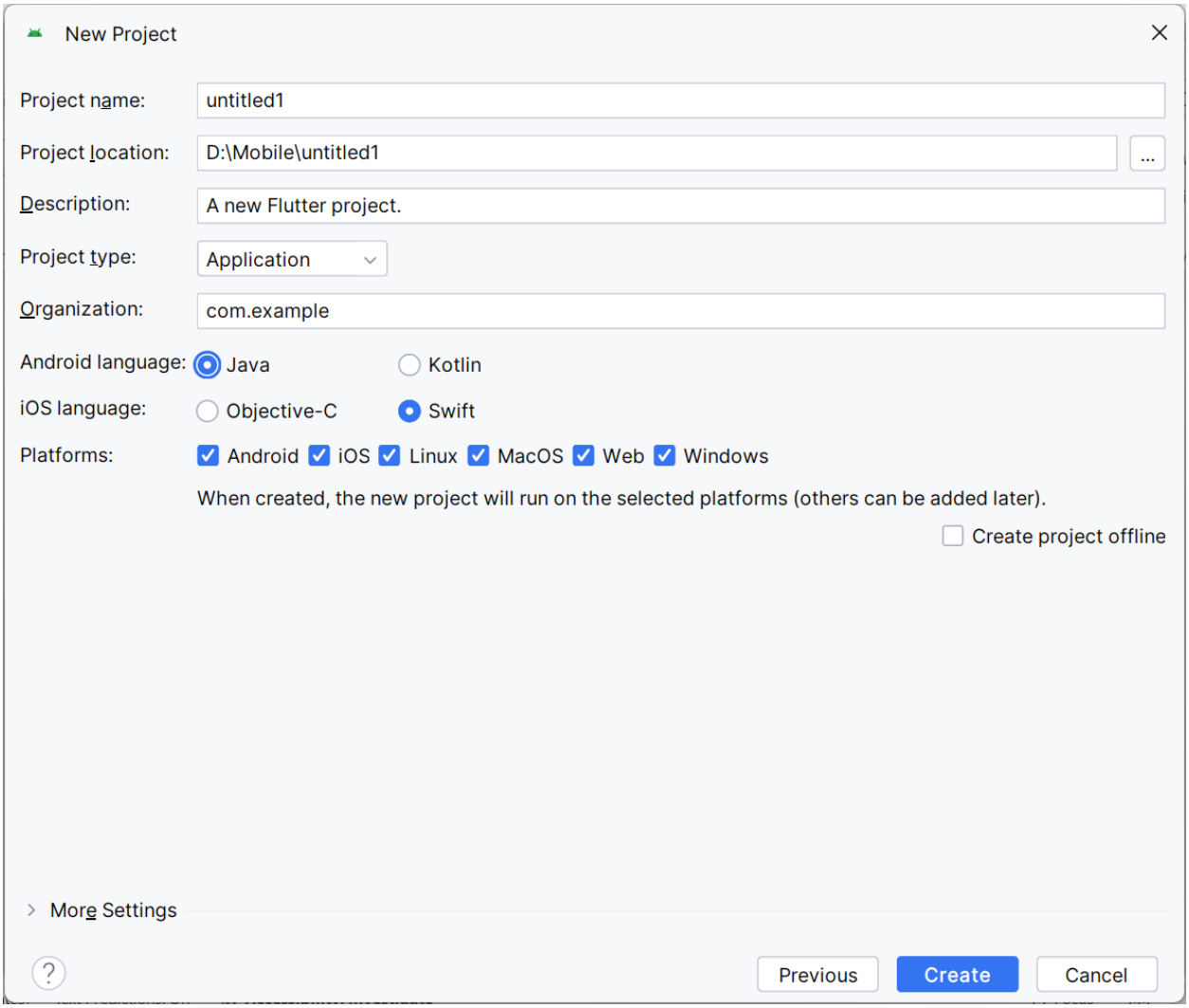
Tạo ứng dụng Flutter với Android Studio:



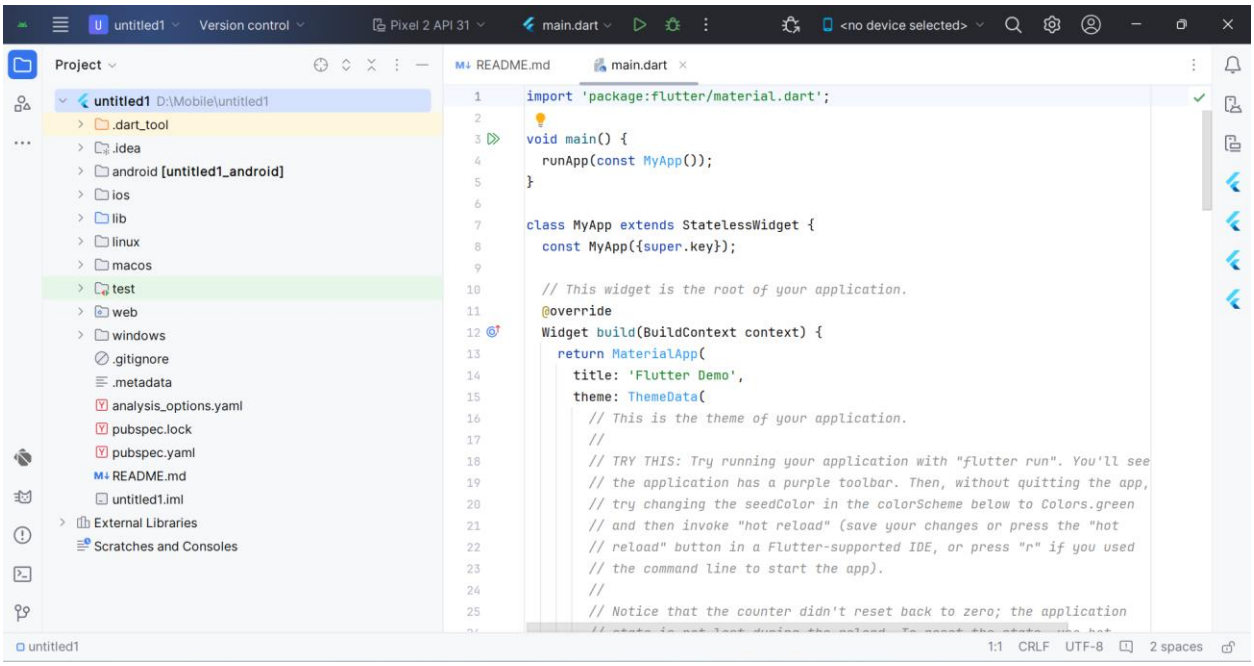
Chọn New Flutter Project



Lưu ý chọn đường dẫn (Path) đã cài đặt Flutter SDK

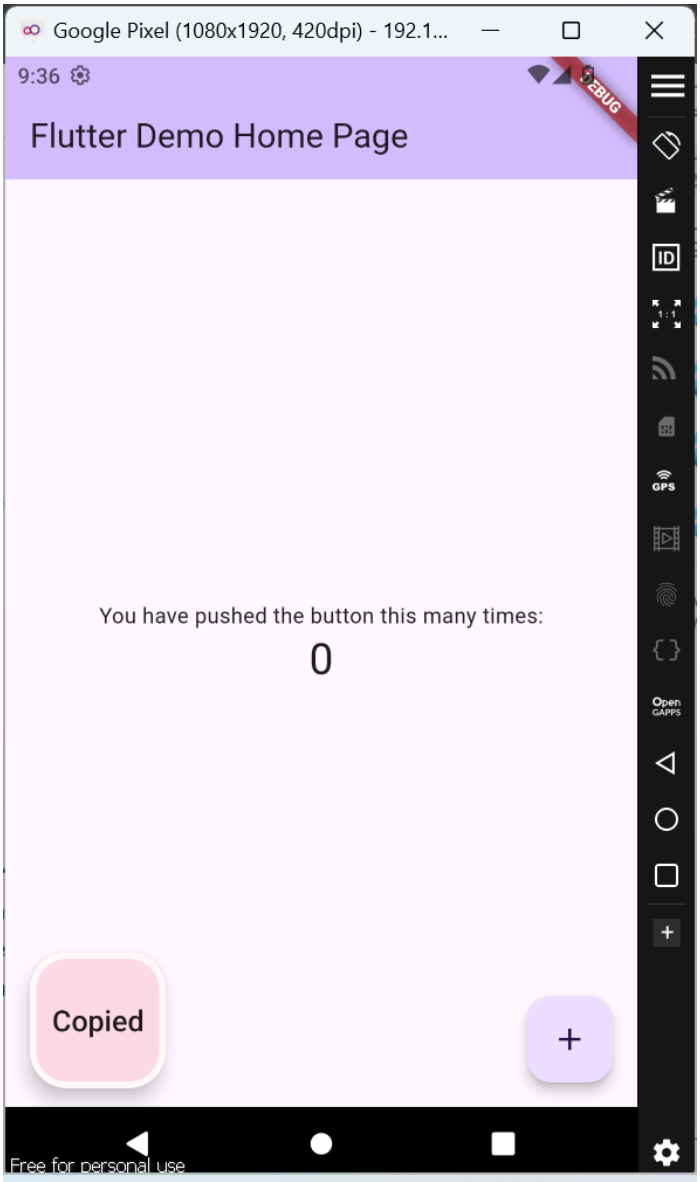


Điền tên project, đường dẫn, công cụ môi trường (chọn một hoặc nhiều nền tảng ứng dụng có thể hỗ trợ) và bấm nút “Create”



Giao diện của project đã được tạo với file code trong main.dart.

Chọn “Run” hoặc Shift+F10 để chạy ứng dụng (lưu ý cần tạo máy ảo trước, có thể dùng các máy ảo đã có của Android Studio)



Giải thích code:

Đoạn code này tạo ra một ứng dụng Flutter cơ bản với một nút bấm và một bộ đếm. Khi người dùng nhấn vào nút bấm, bộ đếm sẽ tăng lên một đơn vị.

```
import 'package:flutter/material.dart';
```

Dòng này nhập gói “flutter/material.dart”, cung cấp các widget và các công cụ thiết kế giao diện người dùng tuân theo ngôn ngữ thiết kế Material của Google.

```
void main() {
  runApp(const MyApp());
```

```
}
```

Hàm main() là điểm khởi đầu của ứng dụng. Hàm runApp nhận vào một widget, ở đây là MyApp, và hiển thị nó lên màn hình. const MyApp() tạo một instance của MyApp với từ khóa const để tối ưu hóa hiệu năng và đảm bảo rằng widget này là bất biến.

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Demo',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
        useMaterial3: true,
      ),
      home: const MyHomePage(title: 'Flutter Demo Home Page'),
    );
  }
}
```

- class MyApp extends StatelessWidget : MyApp là một lớp kế thừa từ StatelessWidget, nghĩa là trạng thái của nó không thay đổi trong suốt vòng đời của widget này.

- const MyApp({super.key}); : Constructor của MyApp, sử dụng từ khóa const để tạo một instance bất biến của MyApp. super.key là tham số được truyền đến constructor của lớp cha StatelessWidget.

- @override : Chỉ thị cho trình biên dịch rằng phương thức build được định nghĩa lại từ lớp cha StatelessWidget.

- Widget build(BuildContext context) : Phương thức build xây dựng và trả về cây widget mà Flutter hiển thị.

- MaterialApp : Widget gốc của ứng dụng, cung cấp nhiều tính năng như điều hướng, chủ đề, và hỗ trợ đa ngôn ngữ. title đặt tiêu đề cho ứng dụng, theme định nghĩa giao diện và chủ đề của ứng dụng.

- ColorScheme.fromSeed(seedColor: Colors.deepPurple) : Định nghĩa bảng màu cho ứng dụng dựa trên màu sắc chính là màu tím đậm.

- useMaterial3: true : Sử dụng thiết kế Material Design phiên bản 3.

- home: const MyHomePage(title: 'Flutter Demo Home Page') : Xác định widget gốc của ứng dụng là MyHomePage.

```
class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});
  final String title;
  @override
  State<MyHomePage> createState() => _MyHomePageState();
}
```

- class MyHomePage extends StatefulWidget : MyHomePage là một lớp kế thừa từ StatefulWidget, nghĩa là trạng thái của nó có thể thay đổi trong suốt vòng đời của widget này.

- const MyHomePage({super.key, required this.title}); : Constructor của MyHomePage, nhận vào một title là tiêu đề của trang.

- final String title; : Thuộc tính title của widget, được khởi tạo một lần và không thể thay đổi.

- @override : Chỉ thị cho trình biên dịch rằng phương thức createState được định nghĩa lại từ lớp cha StatefulWidget.

- State<MyHomePage> createState() => _MyHomePageState(); : Tạo một instance của _MyHomePageState để quản lý trạng thái của widget.

```
class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0;
  void _incrementCounter() {
    setState(() {
      _counter++;
    });
  }
```

- class _MyHomePageState extends State<MyHomePage> : _MyHomePageState là lớp quản lý trạng thái cho MyHomePage.

- int _counter = 0; : Biến đếm _counter, được khởi tạo với giá trị 0.

- void _incrementCounter() { setState(() { _counter++; }); } : Phương thức _incrementCounter tăng giá trị của _counter và gọi setState để cập nhật giao diện người dùng.

```
@override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(
    backgroundColor: Theme.of(context).colorScheme.inversePrimary,
    title: Text(widget.title),
   ),
   body: Center(
    child: Column(
     mainAxisAlignment: MainAxisAlignment.center,
     children: <Widget>[
      const Text('You have pushed the button this many times:'),
      Text('$_counter', style: Theme.of(context).textTheme.headlineMedium),
     ],
    ),
   ),
   floatingActionButton: FloatingActionButton(
    onPressed: _incrementCounter,
    tooltip: 'Increment',
    child: const Icon(Icons.add),
   ),
  );
```

```
  }
}
```

- Widget build(BuildContext context) : Phương thức build xây dựng và trả về widget tree mà Flutter hiển thị.

- Scaffold : Widget cung cấp một cấu trúc giao diện cơ bản như app bar, drawer, floating action button, và các thành phần cơ bản khác của giao diện Material Design.

- AppBar : Thanh công cụ ở đầu màn hình, với backgroundColor là màu chủ đề ngược và title là tiêu đề của trang.

- body: Center : Widget Center căn giữa con của nó theo cả chiều ngang và chiều dọc.

- Column : Widget Column sắp xếp các con của nó theo chiều dọc.

- mainAxisAlignment: MainAxisAlignment.center : Căn giữa các con của Column theo trục dọc.

- const Text('You have pushed the button this many times:') : Widget Text hiển thị một đoạn văn bản cố định.

- Text('$_counter', style: Theme.of(context).textTheme.headlineMedium) : Widget Text hiển thị giá trị của _counter với kiểu chữ headlineMedium từ chủ đề của ứng dụng.

- floatingActionButton: FloatingActionButton : Nút bấm nổi, khi được nhấn sẽ gọi phương thức `_incrementCounter`.

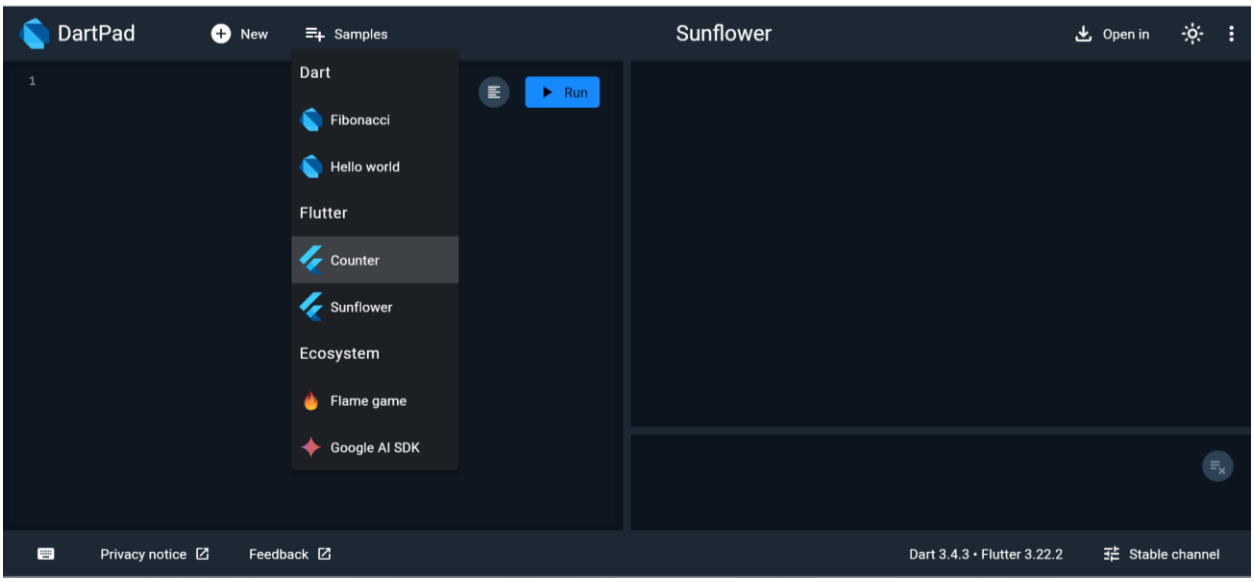
- onPressed: _incrementCounter : Gán phương thức _incrementCounter cho sự kiện onPressed của nút bấm.

- tooltip: 'Increment' : Văn bản hiển thị khi người dùng giữ lâu trên nút bấm.

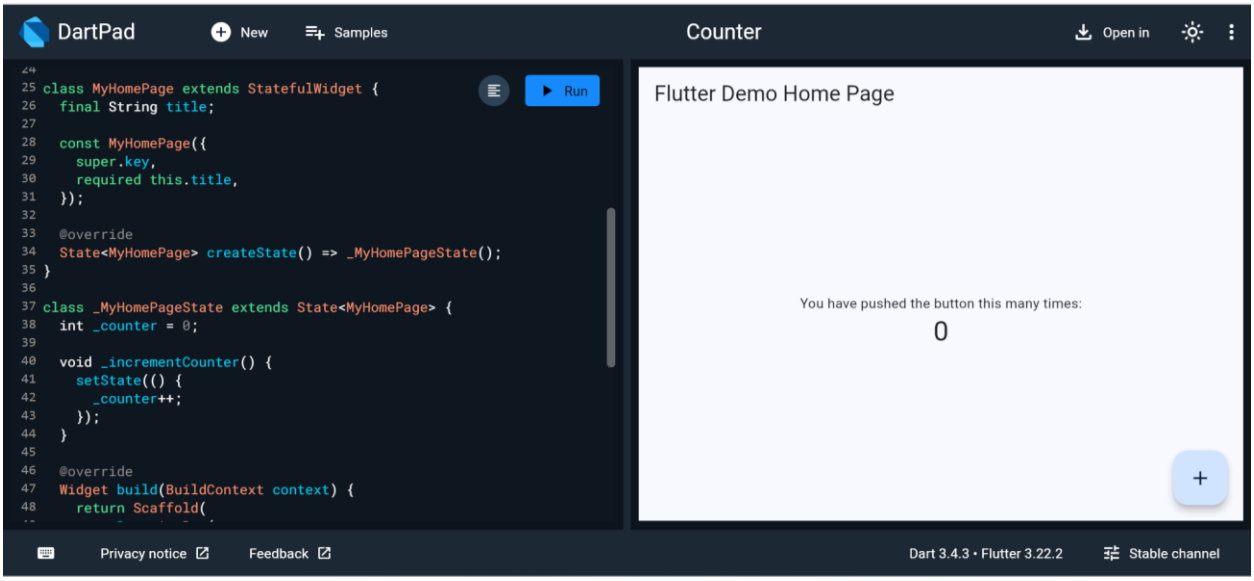
- child: const Icon(Icons.add) : Icon hiển thị trên nút bấm.

Xem thêm các sample mẫu:

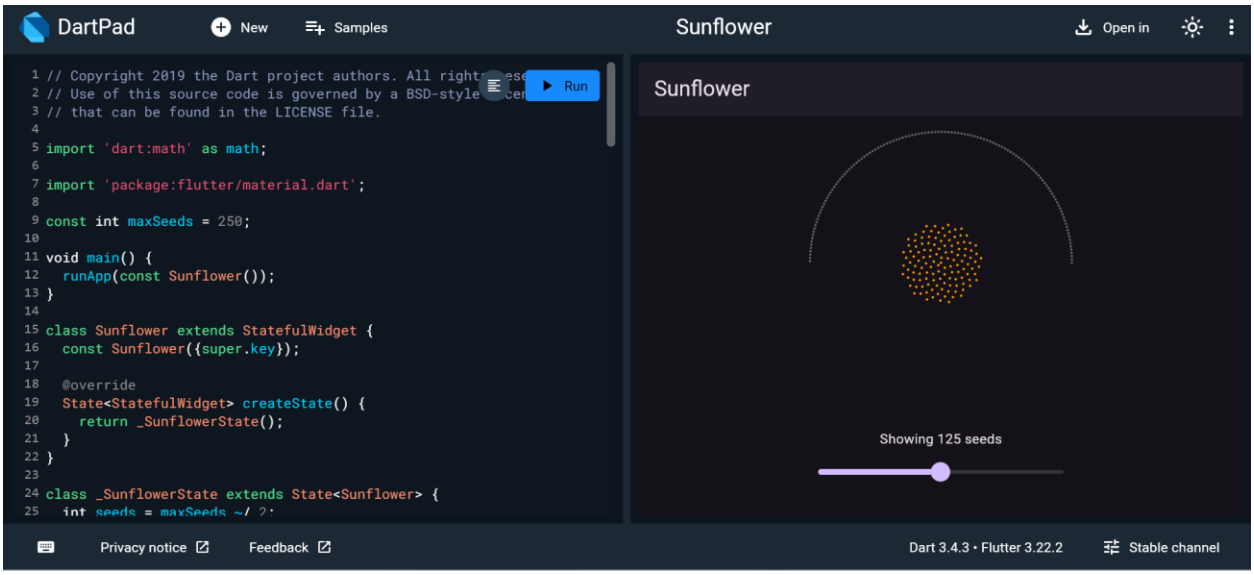
Trong Dartpad chọn Sample



Ứng dụng Counter (chọn Sample → Flutter/ Counter)



Ứng dụng Sunflower (chọn Sample → Flutter/ Sunflower)



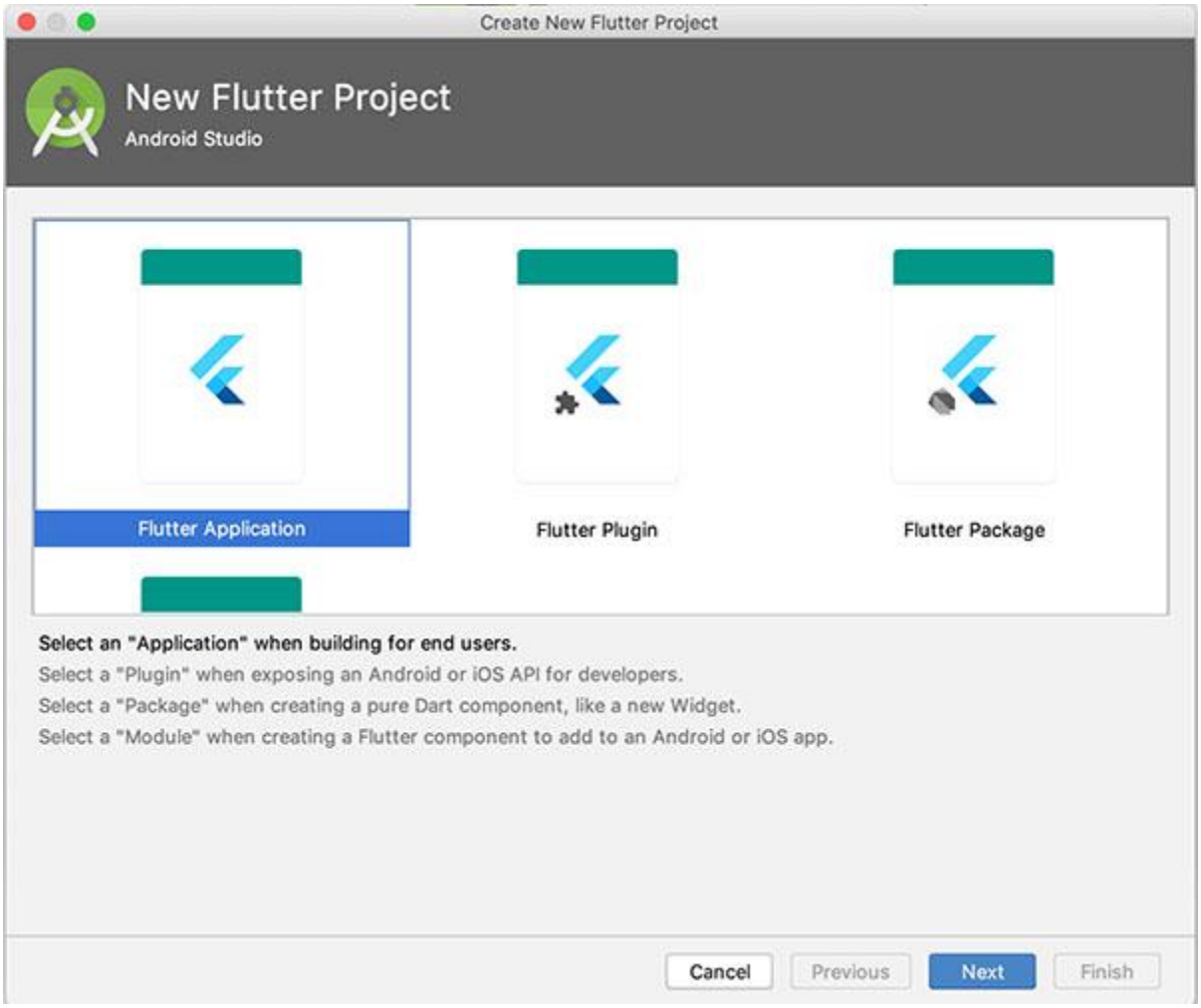
Tạo ứng dụng Flutter với Android Studio

Bài thực hành 1 sẽ hướng dẫn tạo **Flutter Application** đầu tiên trên Android Studio, quan đó giúp ta hiểu cơ bản cấu trúc của một project ứng dụng Flutter

Bước 1 – Mở Android Studio

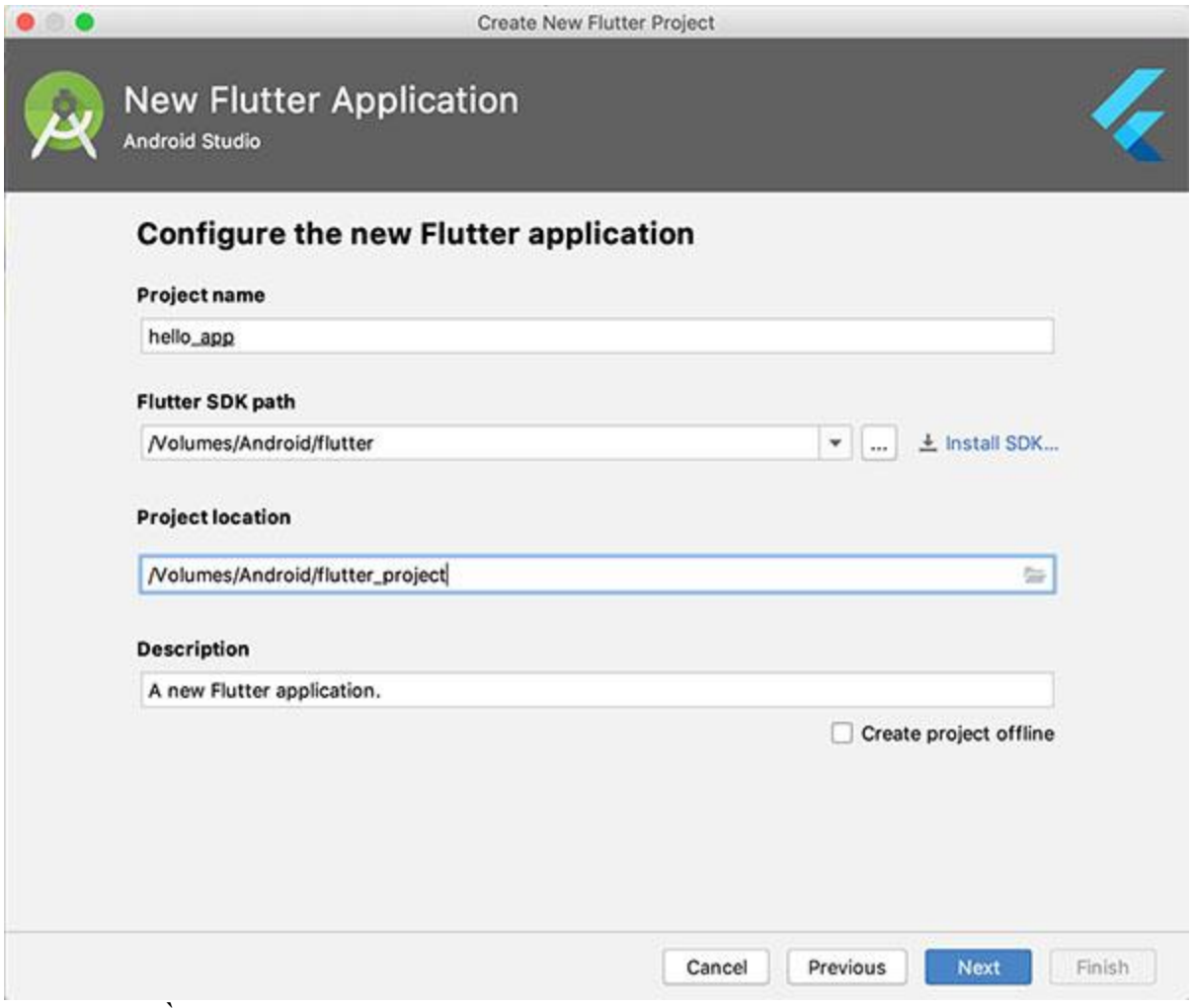
Bước 2 – Tạo Flutter Project mới: Chon **Start a New Flutter Project** hoặc từ menu **File → New → New Flutter Project**

Bước 3 − Có nhiều loại project Flutter khác nhau chúng ta chọn **Flutter Application** và nhấn **Next**.

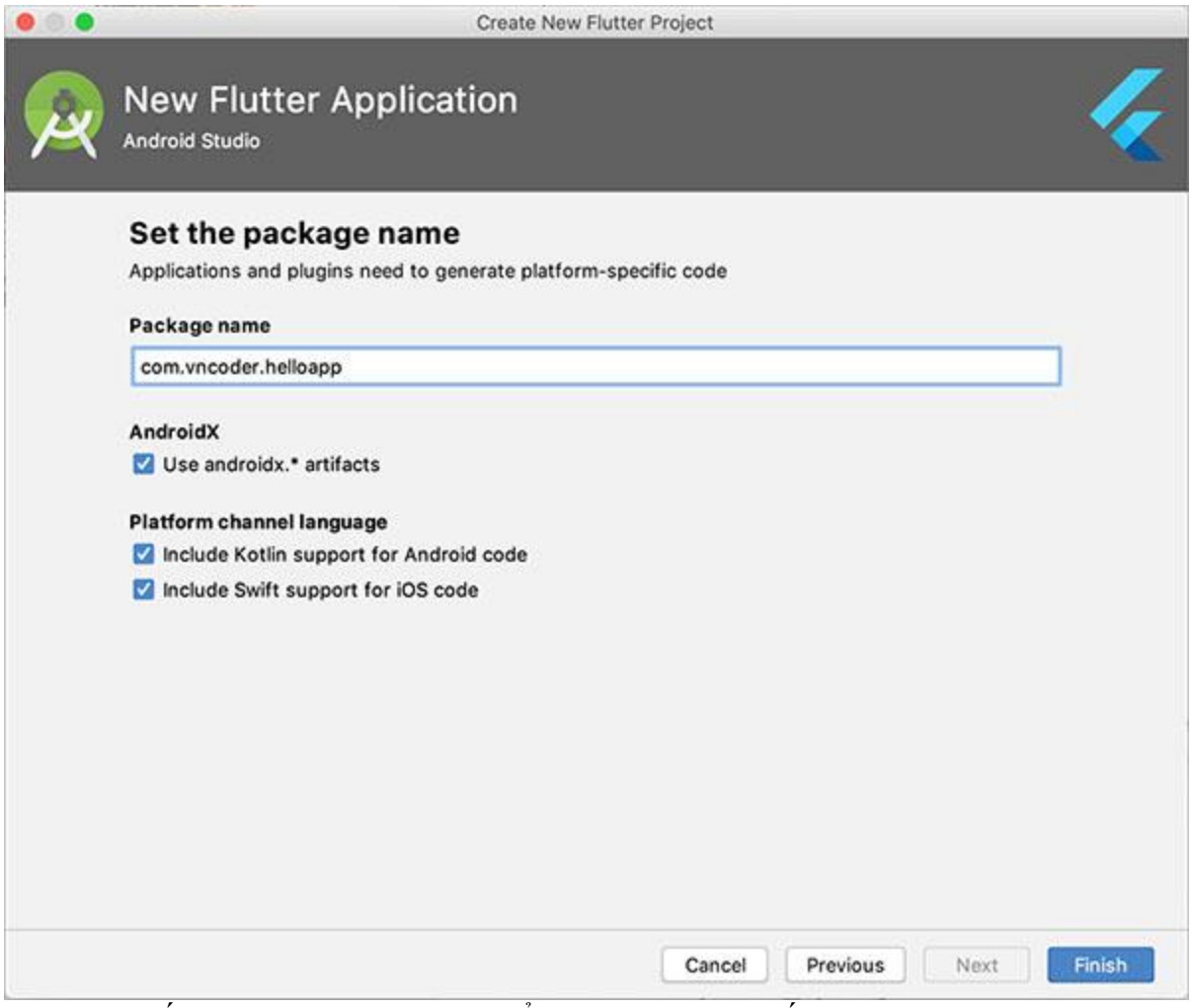


Bước 4 − Đặt tên và mô tả cho Project sau đó chọn **Next**.

Bạn đặt tên cho project là **hello_app** hoặc tên bất kì. Chọn đường dẫn thư mục Flutter SDK, nơi lưu trữ project và mô tả của ứng dụng

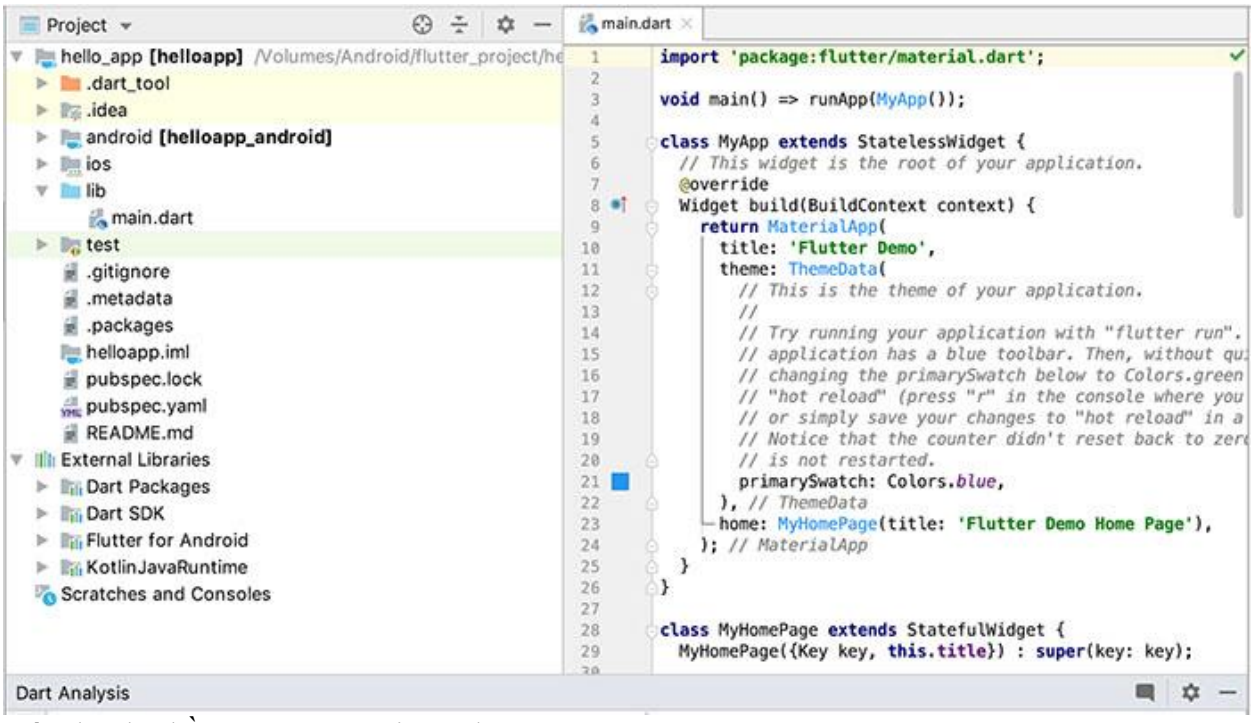


Bước 5 – Điền package_name cho ứng dụng



Bước 6 – Nhấn Finish và đợi một lúc để Android Studio tiến hành việc tạo project

Sau khi tạo xong, chúng ta có thể thấy cấu trúc của một project Flutter như bên dưới



Các thành phần của một Project Flutter

- **android** − Thư mục code sinh tự động cho ứng dụng Android
- **ios** − Thư mục code sinh tự động cho ứng dụng iOS
- **lib** − Main folder chứa Dart code được viết khi sử dụng flutter framework
- **ib/main.dart** − File đầu tiên là điểm khởi đầu của ứng dụng Flutter application
- **test** − Folder chứa Dart code để test flutter application
- **test/widget_test.dart** − Sample code
- **.gitignore** − Git version control file - File này chứa cấu hình cho project git
- **.metadata** − sinh tự động bởi flutter tools
- **.packages** − sinh tự động để theo dõi flutter packages
- **.iml** − project file của Android studio
- **pubspec.yaml** − Được sử dụng **Pub**, Flutter package manager
- **pubspec.lock** − Sinh tự động bởi Flutter package manager, **Pub**
- **README.md** − Project description được viết theo cấu trúc Markdown

Bước 7 − Mặc định Android Studio đã tạo sẵn cho chúng ta code ở *lib/main.dart file* , tuy nhiên chúng ta xoá đi và viết lại đoạn code dưới đây

```
import 'package:flutter/material.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  // This widget is the root of your application.
  @override
```

```
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Hello World Demo Application',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: MyHomePage(title: 'Home page'),
    );
  }
}
class MyHomePage extends StatelessWidget {
  MyHomePage({Key key, this.title}) : super(key: key);
  final String title;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(this.title),
      ),
      body: Center(
        child:
        Text(
          'Hello World',
        )
      ),
    );
  }
}
```

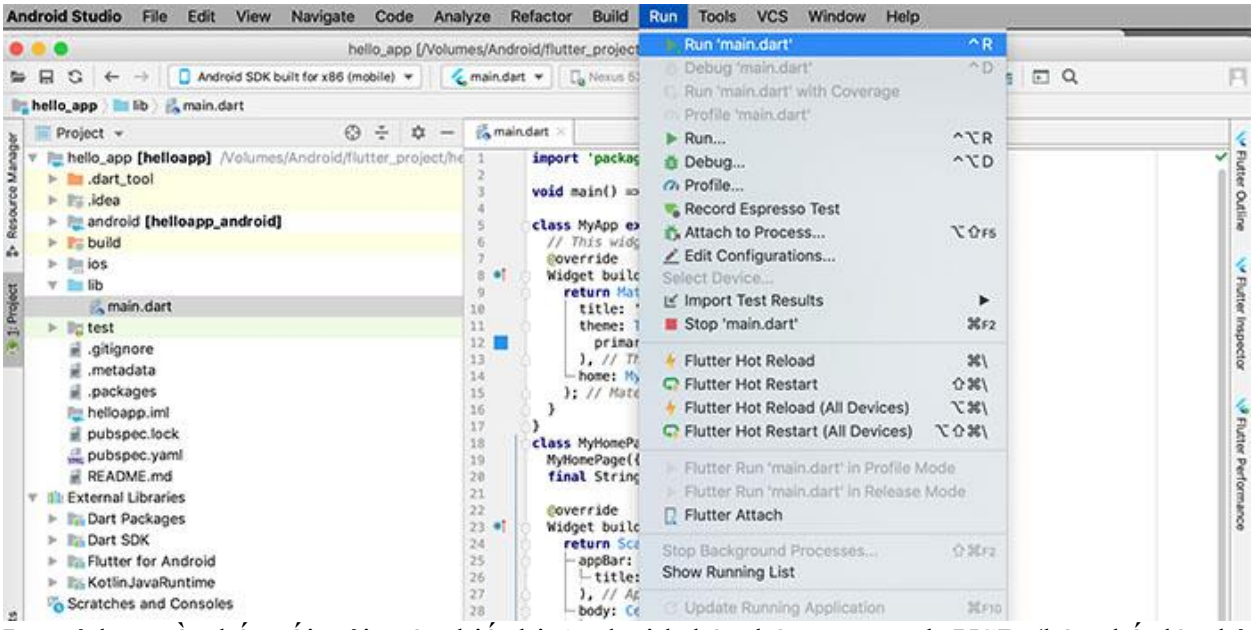
Ý nghĩa chi tiết của đoạn code trên

- **Dòng 1** − import flutter package, tên là *material*. **Material** là một flutter package được sử dụng để tạo giao diện người dùng theo Material design cho Android.
- **Dòng 2** − là điểm khởi đầu của Flutter application là hàm main của ứng dụng. Phương thức *runApp* được gọi và truyền vào đối tượng của lớp *MyApp*. Mục đích chính của phương thức *runApp* là đưa giao diện widget vào hiển thị trên màn hình
- **Dòng 5-17** − Widget được sử dụng để tạo UI (giao diện người dùng) qua flutter framework. *StatelessWidget* là một widget, nó không bao gồm trạng thái nào của widget. *MyApp* kế thừa *StatelessWidget* và ghi đè phương thức *build method*. Mục đích của phương thức *build* là tạo một phần UI cho ứng dụng. Ở đây, phương thức *build* sử dụng *MaterialApp*, một widget để tạo layout UI gốc cho ứng dụng. Bao gồm 3 thành phần chính là - *title (tiêu đề), theme (chủ đề)* và *home (trang chủ*

hay phần màn hình).*title* là tiêu đề của ứng dụng*theme* là chủ đề của widget. Ở đây, chúng ta set chủ đề là *blue* đó là màu sắc chủ đạo của ứng dụng thông qua class *ThemeData* và các thuộc tính của nó ví dụ *primarySwatch*.home phần màn hình của ứng dụng, nó được tạo bởi một widget khác, **MyHomePage**

- **Dòng 19 - 38** – *MyHomePage* tương tự *MyApp* ngoại trừ nó trả về *Scaffold* Widget. *Scaffold* là môt top level widget đứng sau *MaterialApp* widget được sử dụng để tạo UI theo material design. Nó có hai thuộc tính quan trọng nhất là *appBar* để hiển thị phần đầu của ứng dụng và body để hiển thị nôi dung chính của ứng dụng. *AppBar* là một widget khác để tạo phần đầu của ứng dụng và sử dụng các thuộc tính của *appBar*. Trong các thuộc tính của *body*, chúng ta sử dụng một *Center* widget, đây là một widget con. *Text* là một widget cuối cùng, phổ biến để hiển thị một văn bản giữa màn hình

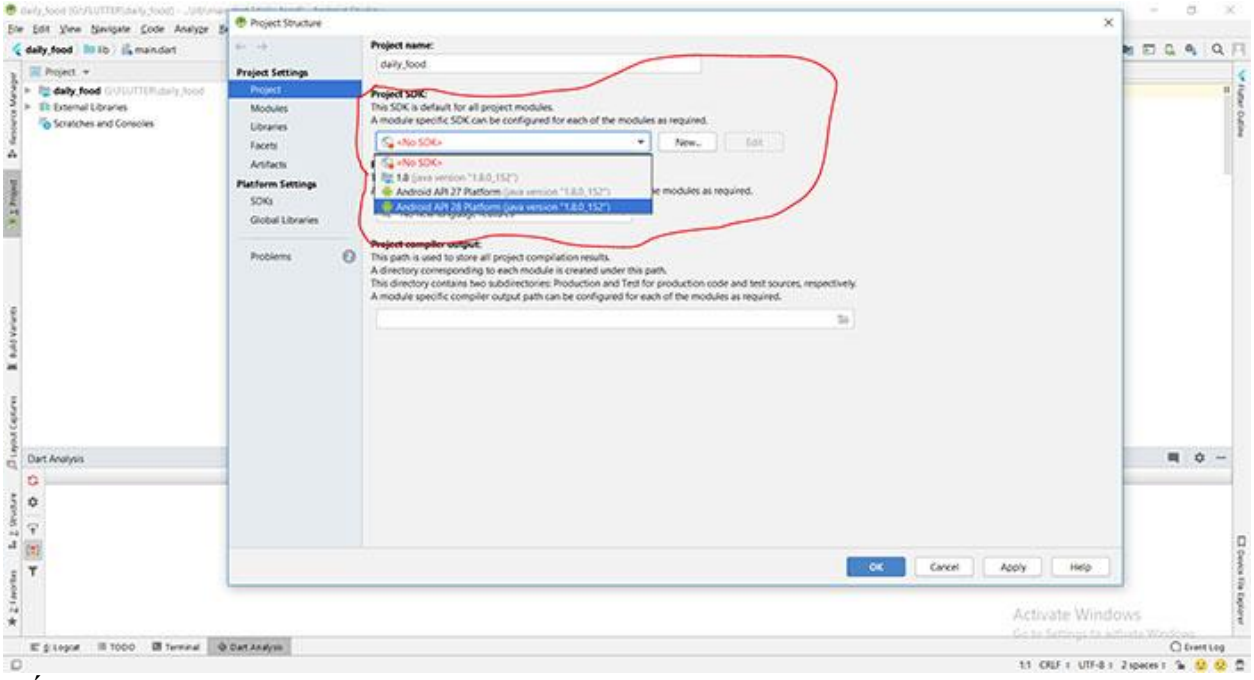
Bước 8 – Bây giờ chúng ta bắt đầu chạy ứng dụng bằng lệnh **Run → Run main.dart**



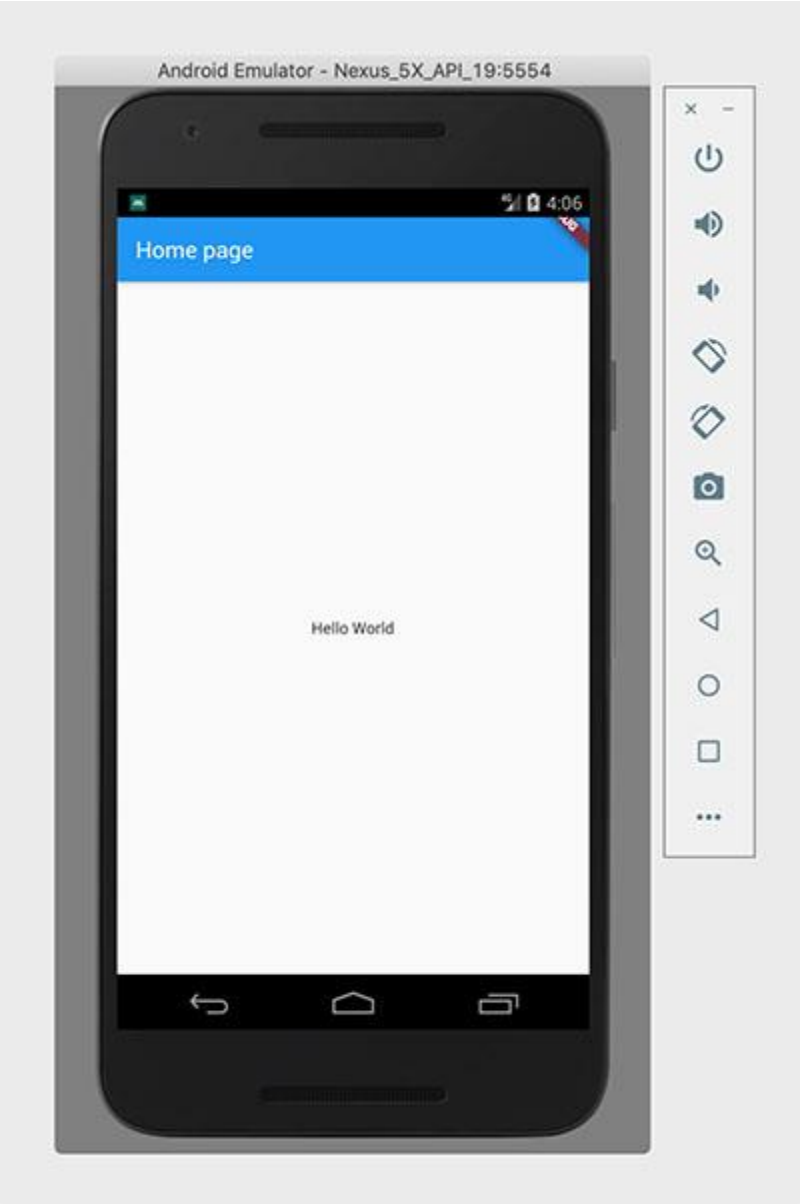
Lưu ý bạn cần kết nối với một thiết bị Android thật thông qua cab USB (bật chế độ nhà phát triển) hoặc kết nối với một máy ảo Android Emulator

Trong trường hợp Android Studio báo lỗi không kết nối được với thiết bị (**flutter run: No connected devices**) có thể là do bạn chưa chọn API Android cho project

Các bạn chọn **File → Project Structure** và thêm vào SDK Android mới nhất



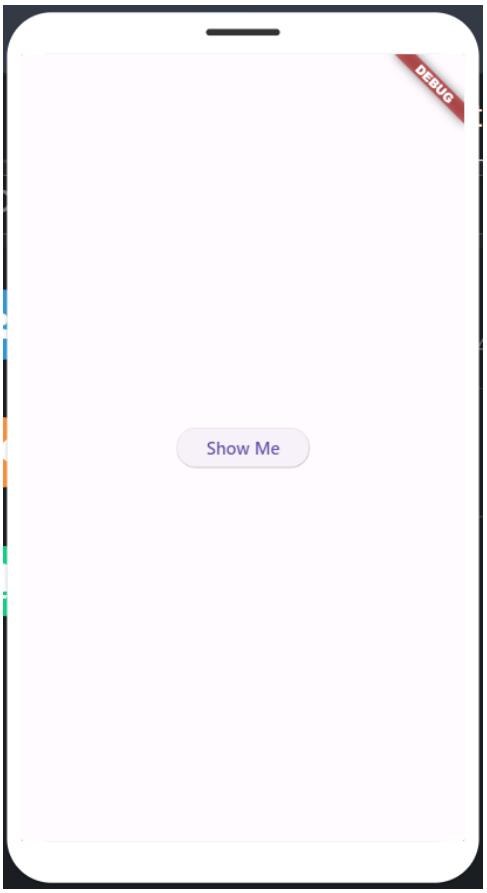
Kết quả khi chạy Project Flutter trên máy ảo Android Emulator

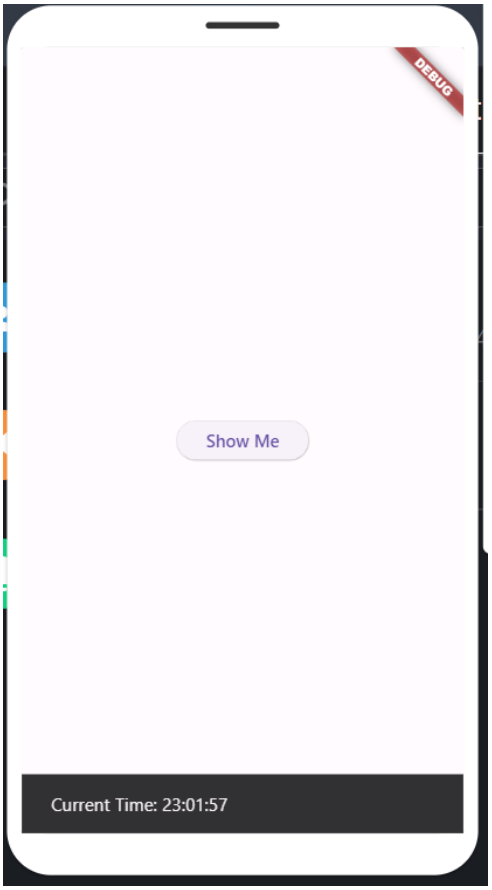


Như vậy ta có thể **tạo ứng dụng Flutter đầu tiên**, giải thích các thành phần chính của project cũng như code của file main.

Bài 1:

Tạo một ứng dụng có giao diện như hình. Ứng dụng có một nút bấm (Button) hiển thị thông báo giờ hiện hành





<u>Hướng dẫn:</u>

Trong main.dart tạo cấu trúc sau:

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      home: MyPage(),
    );
  }
}
```

```
class MyPage extends StatelessWidget {
  const MyPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Center(
        child: ElevatedButton(
            child: const Text('Show Me'),
            onPressed: () {}),
      ),
    );
  }
}
```

Trong đó:

```
ElevatedButton(
            child: const Text('Show Me'),
            onPressed: () {}),
```

là Button có nội dung là “Show Me” và có hàm xử lý sự kiện khi nhấn Button là hàm onPressed

Chạy ứng dụng và xem giao diện hiển thị

Thêm xử lý để khi người dùng nhấn vảo Button sẽ hiện ra ngày giờ hiện hành

```
child: ElevatedButton(
    child: const Text('Show Me'),
    onPressed: () {
      final now = DateTime.now(); // Lấy thời gian hiện hành
      final formattedTime =
          DateFormat('HH:mm:ss').format(now); // Định dạng thời gian
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text('Current Time: $formattedTime'),
        ), // SnackBar
      );
    }), // ElevatedButton
```

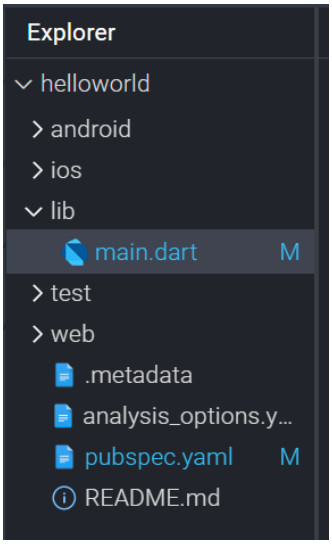
Trong đó SnackBar là thông báo sẽ hiển thị khi người dùng nhấn vào button.

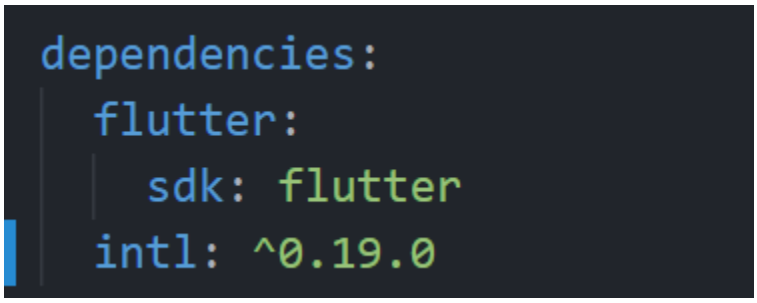
Lưu ý: cần import thêm package

```
import 'package:intl/intl.dart';
```

Nếu có lỗi xảy ra khi thêm gói, cần fix như sau:

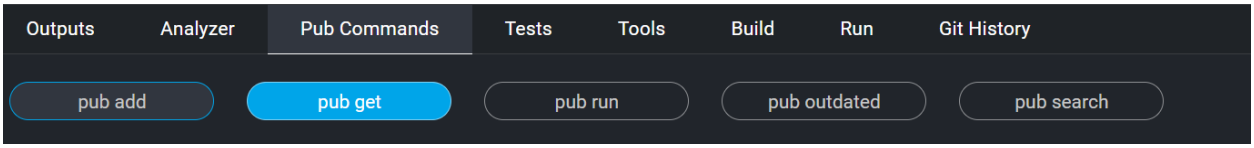
Vào file puspec.yaml, thêm intl: ^0.19.0 như hình



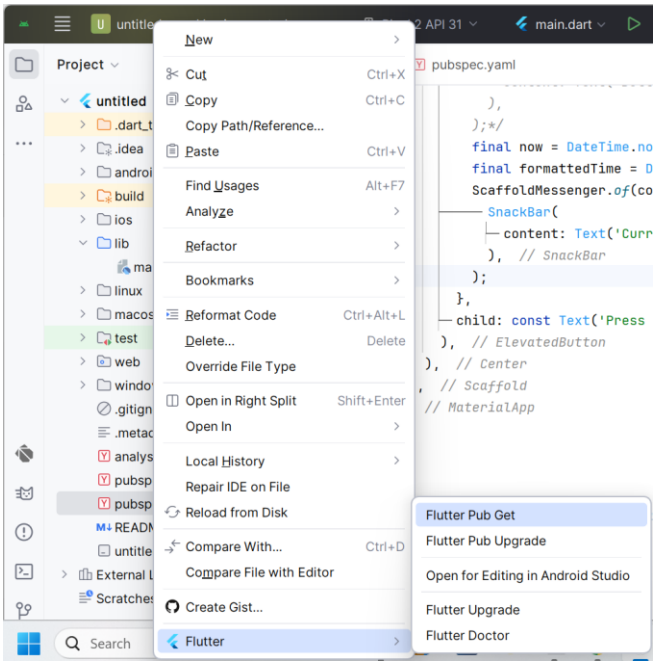


Sau đó chọn chức năng Pub get

Ví dụ sau sử dụng FlutLab.io



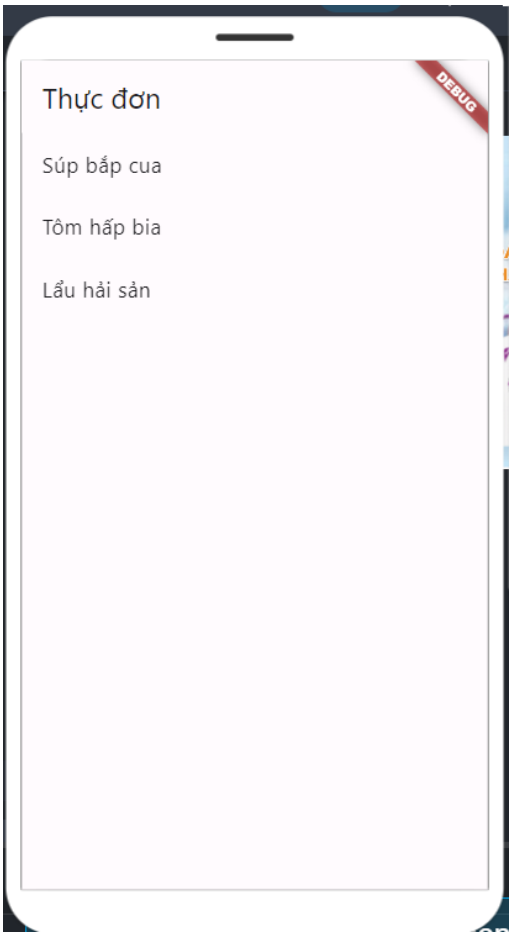
Đối với Android Studio thì chọn Flutter → Flutter Pub Get

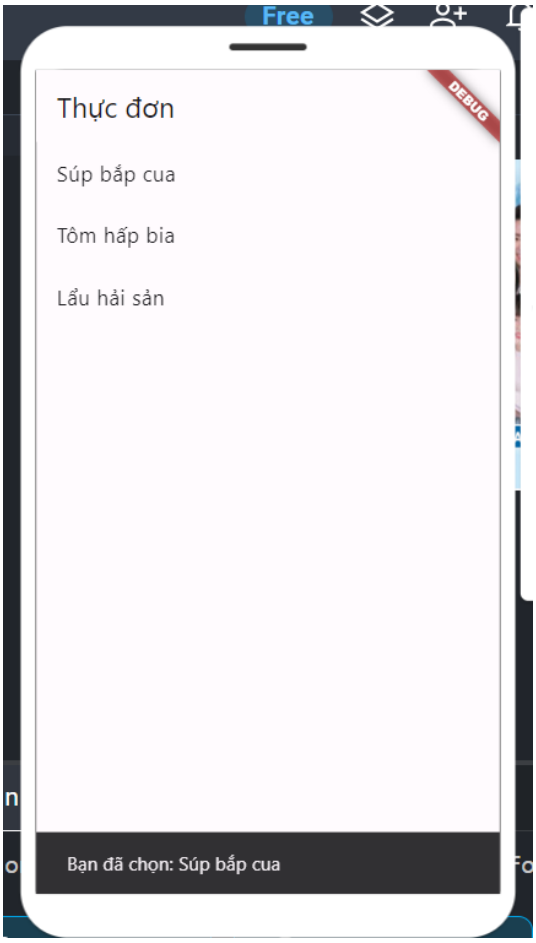


Chạy và xem kết quả.

Bài 2:

Xây dựng ứng dụng với danh sách các món ăn, khi người dùng chọn vào 1 món ăn thì xuất thông báo món ăn vừa được chọn.





<u>Hướng dẫn:</u>

Tạo ứng dụng:

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}
```

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Thực đơn',
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Thực đơn'),
        ), // AppBar
```

```
        body: Builder(
          builder: (BuildContext context) => ListView(
            children: <Widget>[
              ListTile(
                title: const Text('Súp bắp cua'),
              ), // ListTile
              ListTile(
                title: const Text('Tôm hấp bia'),
              ), // ListTile
              ListTile(
                title: const Text('Lẩu hải sản'),
              ), // ListTile
            ], // <Widget>[]
          ), // ListView
        ), // Builder
      ), // Scaffold
    ); // MaterialApp
  }
}
```

Chạy và xem giao diện hiển thị.

Thêm xử lý khi người dùng chọn

```
ListTile(
  title: const Text('Súp bắp cua'),
  onTap: () {
    _showSnackbar(context, 'Súp bắp cua');
  },
), // ListTile
```

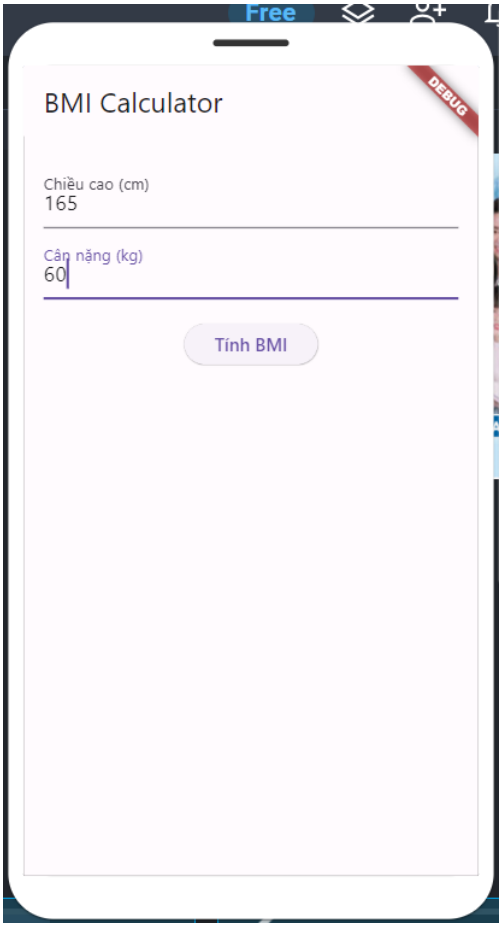
ListTile là các mục trong danh sách. ListTile có một thuộc tính onTap được đặt để gọi hàm _showSnackbar với tên của mục được chọn.
Hàm _showSnackbar nhận BuildContext và một thông báo, sau đó hiển thị một SnackBar với nội dung thông báo đó.

```
  void _showSnackbar(BuildContext context, String message) {
    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text('Bạn đã chọn: $message'),
      ), // SnackBar
    );
  }
}
```

Bài 3:

Viết ứng dụng tính chỉ số BMI cơ thể. Ứng dụng có 2 màn hình. Màn hình thứ nhất cho người dùng nhập vào cân nặng và chiều cao. Người dùng nhấn Button Tính BMI thì ứng dụng hiển thị màn hình thứ hai với kết quả BMI với thông số vừa nhập

BMI = Cân nặng (kg) / (Chiều cao (cm) * Chiều cao (cm))



<u>Hướng dẫn:</u>

Tạo ứng dụng

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'BMI Calculator',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ), // ThemeData
      home: const HomeScreen(),
    ); // MaterialApp
  }
}
```

Màn hình thứ nhất (HomeScreen)

```dart
class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  _HomeScreenState createState() => _HomeScreenState();
}
```

Lớp State để quản lý State cho màn hình thứ nhất

```
class _HomeScreenState extends State<HomeScreen> {
  final TextEditingController _heightController = TextEditingController();
  final TextEditingController _weightController = TextEditingController();
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('BMI Calculator'),
      ), // AppBar
```

Hai ô nhập liệu chiều cao và cân nặng

```
body: Padding(
  padding: const EdgeInsets.all(16.0),
  child: Column(
    children: <Widget>[
      TextField(
        controller: _heightController,
        keyboardType: TextInputType.number,
        decoration: const InputDecoration(
          labelText: 'Chiều cao (cm)',
        ), // InputDecoration
      ), // TextField
      TextField(
        controller: _weightController,
        keyboardType: TextInputType.number,
        decoration: const InputDecoration(
          labelText: 'Cân nặng (kg)',
        ), // InputDecoration
      ), // TextField
```

Button Tính BMI

```
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: () {
              },
              child: const Text('Tính BMI'),
            ), // ElevatedButton
          ], // <Widget>[]
        ), // Column
      ), // Padding
    ); // Scaffold
  }
}
```

Chạy và kiểm tra giao diện

Tạo màn hình thứ hai

```
class ResultScreen extends StatelessWidget {
  final double bmi;

  const ResultScreen({super.key, required this.bmi});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Kết quả BMI'),
      ), // AppBar
      body: Center(
        child: Text(
          'Chỉ số BMI của bạn là: ${bmi.toStringAsFixed(2)}',
          style: const TextStyle(fontSize: 24),
        ), // Text
      ), // Center
    ); // Scaffold
  }
}
```

Viết xử lý cho Button

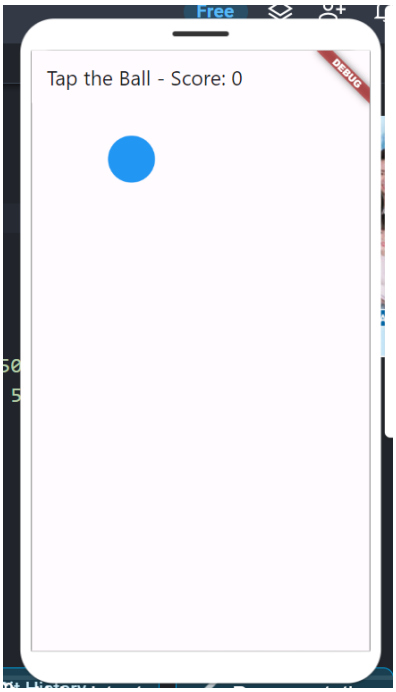
```
onPressed: () {
  double height = double.parse(_heightController.text) / 100;
  double weight = double.parse(_weightController.text);
  double bmi = weight / (height * height);

  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (context) => ResultScreen(bmi: bmi),
    ), // MaterialPageRoute
  );
},
```

Trong đó Navigator là điều hướng sang màn hình thứ hai (ResultScreen)

Chạy ứng dụng và xem kết quả

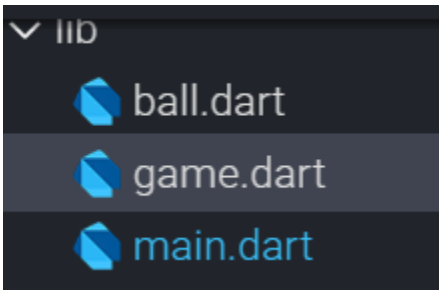
Bài 4:

Viết ứng dụng game chạm bóng đơn giản như sau. Bóng sẽ đi chuyển ngẫu nhiên trên màn hình, nếu người dùng chạm vào bóng thì điểm sẽ tăng 1.



<u>Hướng dẫn:</u>

Tạo cấu trúc project gồm 3 file:



File ball.dart chứa lớp Ball có tọa độ x,y và hàm onTap

```
import 'package:flutter/material.dart';

class Ball extends StatelessWidget {
  final double x;
  final double y;
  final Function onTap;

  Ball({required this.x, required this.y, required this.onTap});

  @override
  Widget build(BuildContext context) {
```

```
    return Positioned(
      left: x,
      top: y,
      child: GestureDetector(
        onTap: () => onTap(),
        child: Container(
          width: 50,
          height: 50,
          decoration: BoxDecoration(
            color: Colors.blue,
            shape: BoxShape.circle,
          ),
        ),
      ),
    );
  }
}
```

Lớp Ball sẽ quản lý bóng có hình tròn màu xanh.

Widget build(BuildContext context): Đây là phương thức build của một widget. Phương thức này tạo ra một widget tree để mô tả cách mà widget này sẽ được hiển thị trên màn hình.

Positioned: là một widget dùng để định vị widget con của nó tương đối với các cạnh của một widget cha Stack. Các tham số left: x và top: y xác định vị trí của widget con (GestureDetector trong trường hợp này) từ cạnh trái và cạnh trên của widget cha.

GestureDetector: là một widget dùng để nhận các sự kiện tương tác của người dùng, chẳng hạn như nhấn, kéo, hoặc vuốt. Tham số onTap: () => onTap() định nghĩa hành động sẽ được thực hiện khi widget con bị nhấn (tap). Ở đây, onTap là một hàm được gọi khi người dùng nhấn vào widget con.

Container: là một widget tiện ích dùng để chứa và định dạng các widget con của nó. Ở đây, Container có chiều rộng (width) và chiều cao (height) là 50.

BoxDecoration: decoration: BoxDecoration(...) định nghĩa kiểu trang trí của Container.

color: Colors.blue đặt màu nền của Container là màu xanh dương.

shape: BoxShape.circle làm cho Container có hình dạng là hình tròn.

Sau đó, trong file game.dart, tạo lớp Game

```
import 'dart:async';
import 'dart:math';
import 'package:flutter/material.dart';
import 'ball.dart';

class Game extends StatefulWidget {
  @override
  _GameState createState() => _GameState();
}
```

Lớp GameState để quản lý trạng thái của Game

```
class _GameState extends State<Game> {
  double x = 0.0;
  double y = 0.0;
  int score = 0;
  late Timer timer;
  Random random = Random();

  @override
  void initState() {
    super.initState();
    startGame();
  }
```

Xử lý khi khởi chạy game và xử lý khi chạm bóng

```
void startGame() {
    timer = Timer.periodic(Duration(milliseconds: 1000), (timer) {
      setState(() {
```

```
        x = random.nextDouble() * (MediaQuery.of(context).size.width -
50);
        y = random.nextDouble() * (MediaQuery.of(context).size.height
- 50);
      });
    });
  }

  void onBallTapped() {
    setState(() {
      score++;
    });
  }
```

Khi khởi tạo Game ban đầu:

timer = Timer.periodic(Duration(milliseconds: 1000), (timer) { ... });:

Timer.periodic là một hàm tạo ra một bộ đếm thời gian định kỳ. Nó sẽ thực hiện một hành động nào đó sau mỗi khoảng thời gian đã định.

Duration(milliseconds: 1000) định nghĩa khoảng thời gian là 1000 mili giây (1 giây).

(timer) { ... } là một hàm ẩn danh (anonymous function) được gọi mỗi khi bộ đếm thời gian kích hoạt. Trong trường hợp này, hàm này sẽ được gọi mỗi giây một lần.

setState(() { ... });:

setState là một phương thức trong StatefulWidget được sử dụng để thông báo cho Flutter rằng trạng thái của widget đã thay đổi và cần được vẽ lại.

Bên trong setState, các thay đổi về trạng thái sẽ được thực hiện và sau đó widget sẽ được xây dựng lại (rebuild).

x = random.nextDouble() * (MediaQuery.of(context).size.width - 50);:

random.nextDouble() tạo ra một số ngẫu nhiên trong khoảng từ 0.0 đến 1.0.

MediaQuery.of(context).size.width trả về chiều rộng của màn hình.

MediaQuery.of(context).size.width - 50 đảm bảo rằng giá trị x nằm trong khoảng từ 0 đến chiều rộng của màn hình trừ đi 50, để đảm bảo rằng đối tượng không bị vượt ra ngoài biên của màn hình.

x được gán giá trị mới là một vị trí ngẫu nhiên trong khoảng chiều rộng của màn hình.

y = random.nextDouble() * (MediaQuery.of(context).size.height - 50);:

Tương tự như trên, random.nextDouble() tạo ra một số ngẫu nhiên trong khoảng từ 0.0 đến 1.0.

MediaQuery.of(context).size.height trả về chiều cao của màn hình.

MediaQuery.of(context).size.height - 50 đảm bảo rằng giá trị y nằm trong khoảng từ 0 đến chiều cao của màn hình trừ đi 50.

y được gán giá trị mới là một vị trí ngẫu nhiên trong khoảng chiều cao của màn hình.

Khi Bóng được chạm thì cập nhật trạng thái và tăng score thêm 1 điểm

```
void onBallTapped() {
    setState(() {
      score++;
    });
  }
```

Thêm giao diện hiển thị Score điểm số

```
void dispose() {
  timer.cancel();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('Tap the Ball - Score: $score'),
    ), // AppBar
    body: Stack(
      children: [
        Ball(x: x, y: y, onTap: onBallTapped),
      ],
    ), // Stack
  ); // Scaffold
}
```

Trong file main.dart

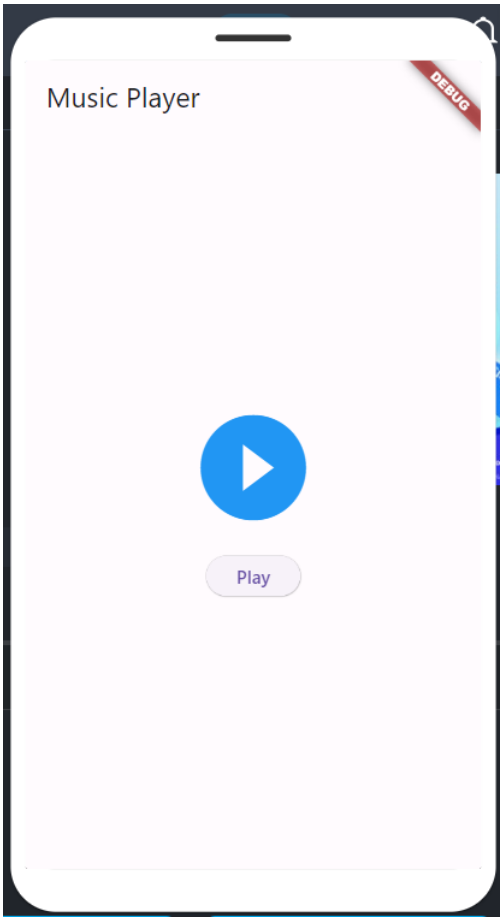
```dart
import 'game.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Tap the Ball Game',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ), // ThemeData
      home: Game(),
    ); // MaterialApp
  }
}
```

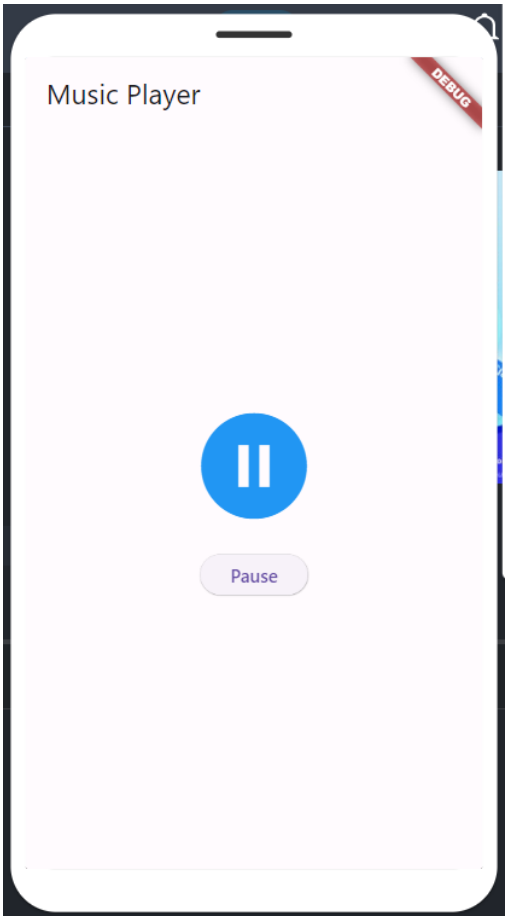
Chạy và xem kết quả

Bóng di chuyển nhanh chậm có thể điều chỉnh thời gian trong hàm startGame

Bài 5:

Tạo ứng dụng nghe nhạc đơn giản





<u>Hướng dẫn:</u>

Tạo project

```
import 'package:flutter/material.dart';
import 'package:audioplayers/audioplayers.dart';

void main() {
  runApp(const MyApp());
}
```

Trong đó cần sửa lại pubspec.yaml

```
dependencies:
  flutter:
    sdk: flutter
  audioplayers: ^0.20.1
```

Sau đó chạy lệnh pub get để update

Lớp MyApp

```
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Music Player',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ), // ThemeData
      home: const MusicPlayerScreen(),
    ); // MaterialApp
  }
}
```

Màn hình nghe nhạc

```
class MusicPlayerScreen extends StatefulWidget {
  const MusicPlayerScreen({super.key});

  @override
  _MusicPlayerScreenState createState() => _MusicPlayerScreenState();
}
```

Lớp quản lý trạng thái của màn hình nghe nhạc

```
class _MusicPlayerScreenState extends State<MusicPlayerScreen> {
  final AudioPlayer _audioPlayer = AudioPlayer();
  bool _isPlaying = false;
  String _currentSong =
      'https://www.soundhelix.com/examples/mp3/SoundHelix-Song-1.mp3';

  void _playPause() {
    if (_isPlaying) {
      _audioPlayer.pause();
    } else {
      _audioPlayer.play(_currentSong);
    }

    setState(() {
      _isPlaying = !_isPlaying;
    });
  }
```

Trong đó

Lớp _MusicPlayerScreenState thừa kế từ State<MusicPlayerScreen> cho phép quản lý trạng thái của MusicPlayerScreen.
Tên lớp có dấu gạch dưới (_) đứng đầu, điều này có nghĩa là lớp này là lớp riêng tư (private) và chỉ có thể sử dụng trong file hiện tại.

_audioPlayer: Một đối tượng AudioPlayer được sử dụng để phát nhạc.
_isPlaying: Biến boolean để theo dõi trạng thái hiện tại của việc phát nhạc (đang phát hoặc dừng).
_currentSong: Chuỗi URL của bài hát hiện tại. (Sv có thể thay đổi link)

Phương thức _playPause được gọi khi nút Play/Pause được nhấn.

Nếu nhạc đang phát (_isPlaying là true), nhạc sẽ tạm dừng bằng cách gọi _audioPlayer.pause().
Nếu nhạc đang dừng (_isPlaying là false), nhạc sẽ được phát bằng cách gọi _audioPlayer.play(_currentSong).
setState được gọi để thay đổi trạng thái của _isPlaying, điều này sẽ làm cho giao diện người dùng được cập nhật lại để phản ánh trạng thái mới.

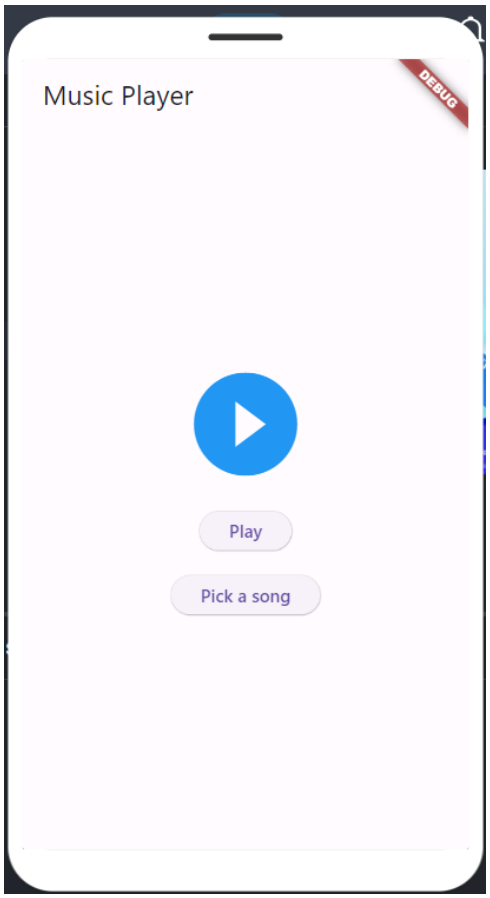
Phương thức build để tạo giao diện nghe nhạc:

```
 Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Music Player'),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Icon(
              _isPlaying ? Icons.pause_circle_filled :
Icons.play_circle_filled,
              size: 100,
              color: Colors.blue,
            ),
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: _playPause,
              child: Text(_isPlaying ? 'Pause' : 'Play'),
            ),
          ],
        ),
      ),
    );
  }
}
```

Giao diện gồm 1 icon (hình ảnh) và 1 Button “Play” khi nhạc đang phát thì là nút “Pause”. Người dùng sẽ nhấn để tủy chỉnh nghe hay tạm dừng nhạc.

Chạy chương trình và xem kết quả.

Cập nhật ứng dụng thêm chức năng chọn nghe file nhạc lưu trong điện thoại



<u>Hướng dẫn:</u>

```
import 'package:file_picker/file_picker.dart';
```

Update pubspec.yaml

```
dependencies:
  flutter:
    sdk: flutter
  audioplayers: ^0.20.1
  file_picker: ^4.0.0
```

Thêm nút chọn File bên dưới nút Play

```
const SizedBox(height: 20),
ElevatedButton(
  onPressed: _playPause,
  child: Text(_isPlaying ? 'Pause' : 'Play'),
), // ElevatedButton
const SizedBox(height: 20),
ElevatedButton(
  onPressed: _pickFile,
  child: const Text('Pick a song'),
), // ElevatedButton
```

Viết hàm xử lý khi nhấn Button chọn file

```
void _pickFile() async {
  FilePickerResult? result =
      await FilePicker.platform.pickFiles(type: FileType.audio);

  if (result != null) {
    setState(() {
      _currentSong = result.files.single.path;
    });
  }
}
```

phương thức _pickFile để mở trình chọn file và cho phép người dùng chọn file nhạc.

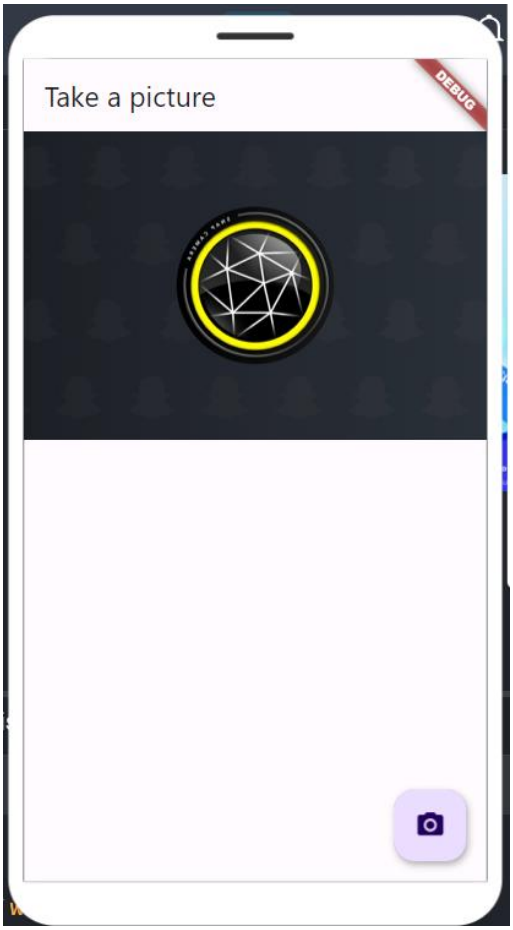
FilePickerResult? result = await FilePicker.platform.pickFiles(type: FileType.audio); sẽ mở trình chọn file và chỉ cho phép chọn các file âm thanh.

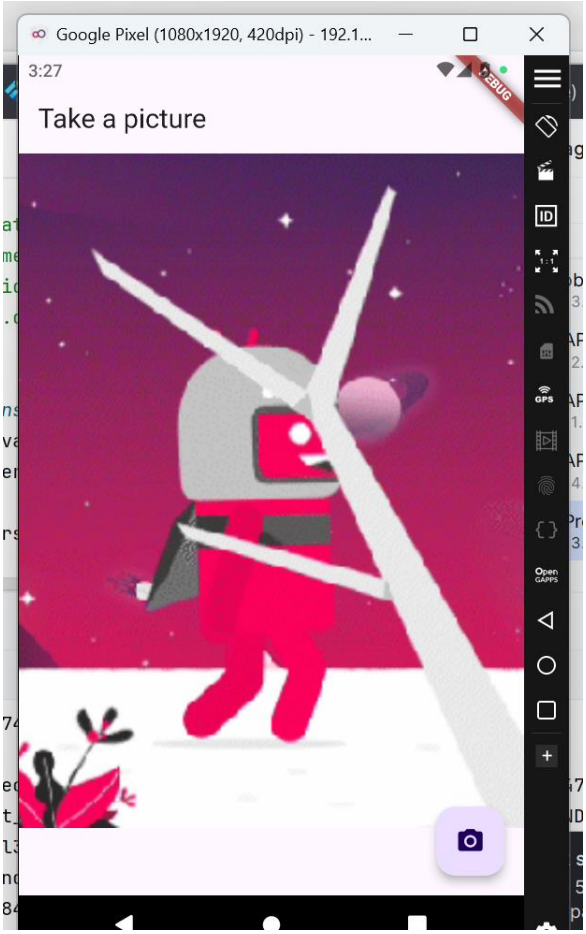
Sau khi chọn file, đường dẫn của file nhạc được lưu vào biến _currentSong.

Chạy và xem kết quả

Bài 6:

Viết ứng dụng mở camera chụp hình, lưu lại và hiển thị hình ảnh.





<u>Hướng dẫn:</u>

Cập nhật pubspec

```
dependencies:
  flutter:
    sdk: flutter
  camera: ^0.10.0+4
  path_provider: ^2.0.11
  path: ^1.8.0
```

Tạo ứng dụng

```
import 'dart:io';
```

```dart
import 'package:flutter/material.dart';
import 'package:camera/camera.dart';
import 'package:path_provider/path_provider.dart';
import 'package:path/path.dart' show join;

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  final cameras = await availableCameras();
  final firstCamera = cameras.first;

  runApp(MyApp(camera: firstCamera));
}
```

Lớp MyApp

```dart
class MyApp extends StatelessWidget {
  final CameraDescription camera;

  const MyApp({super.key, required this.camera});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Camera Demo',
      home: TakePictureScreen(camera: camera),
    );
  }
}
```

Màn hình chụp ảnh

```dart
class TakePictureScreen extends StatefulWidget {
  final CameraDescription camera;

  const TakePictureScreen({super.key, required this.camera});

  @override
  TakePictureScreenState createState() => TakePictureScreenState();
}
```

```
class TakePictureScreenState extends State<TakePictureScreen> {
  late CameraController _controller;
  late Future<void> _initializeControllerFuture;

  @override
  void initState() {
    super.initState();
    _controller = CameraController(
      widget.camera,
      ResolutionPreset.medium,
    );
    _initializeControllerFuture = _controller.initialize();
  }

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  Future<void> _takePicture() async {
    try {
      await _initializeControllerFuture;
      final image = await _controller.takePicture();

      if (!mounted) return;

      final directory = await getApplicationDocumentsDirectory();
      final imagePath = join(directory.path, '${DateTime.now()}.png');
      await image.saveTo(imagePath);

      Navigator.push(
        context,
        MaterialPageRoute(
          builder: (context) => DisplayPictureScreen(imagePath:
imagePath),
        ),
      );
```

```
    } catch (e) {
      print(e);
    }
  }
```

TakePictureScreen nhận CameraDescription và khởi tạo TakePictureScreenState.

CameraController: Điều khiển camera và độ phân giải (ResolutionPreset.medium).

Future<void> _initializeControllerFuture: Chờ camera khởi tạo.

_takePicture(): Chụp hình và lưu trữ. Chờ khởi tạo camera. Chụp hình và lưu vào bộ nhớ. Chuyển sang màn hình hiển thị hình ảnh.

DisplayPictureScreen: Hiển thị hình ảnh đã chụp.

Image.file(File(imagePath)): Hiển thị hình ảnh từ đường dẫn.

Màn hình DisplayPictureScreen

```
class DisplayPictureScreen extends StatelessWidget {
  final String imagePath;

  const DisplayPictureScreen({super.key, required this.imagePath});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Display the Picture')),
      body: Image.file(File(imagePath)),
    );
  }
}
```

Chạy và xem kết quả

Lưu ý: các ứng dụng theo chức năng này phụ thuộc vào thiết bị và emulator